Quickstarts

Quickstart: Bridge with Circle Wallets

Copy page

Use App Kit to bridge USDC between blockchains supported by the Circle Wallets adapter

This quickstart helps you write a server-side script that bridges USDC between blockchains using the Circle Wallets adapter. The examples in this quickstart use Solana Devnet and Arc Testnet, but you can use any blockchain the Circle Wallets adapter **supports** as the source or destination.

Prerequisites

Before you begin, ensure that you've:

- Installed **Node.js v22+**.
- Obtained a **Circle API Key** and **Entity Secret** from the **Circle Console**.
- Created developer-controlled wallets on Solana Devnet and Arc Testnet using the Circle Console.
- Funded your wallets with testnet tokens:
  - Get testnet USDC from the **Circle Faucet**.
  - Get test native tokens from the **Console Faucet**.

Step 1. Set up the project

This step shows you how to prepare your project and environment.

1.1. Set up your development environment

Create a new directory and install App Kit with the Circle Wallets adapter and supporting tools:

Shell

```
# Set up your directory and initialize the project
mkdir app-kit-quickstart-bridge-circle-wallets-adapter
cd app-kit-quickstart-bridge-circle-wallets-adapter
npm init -y

# Install App Kit, Circle Wallets adapter, and tools
npm install @circle-fin/app-kit @circle-fin/adapter-circle-wallets typescript
```

Only need to bridge and want a lighter install than the full App Kit? Install the standalone bridge package instead: `@circle-fin/bridge-kit`

1.2. Configure TypeScript (optional)

This step is optional. It helps prevent missing types in your IDE or editor.

Create a `tsconfig.json` file:

Shell

```
npx tsc --init
```

Then, update the `tsconfig.json` file:

Shell

```
cat <<'EOF' > tsconfig.json
{
  "compilerOptions": {
    "target": "ESNext",
    "module": "ESNext",
    "moduleResolution": "bundler",
    "strict": true,
    "types": ["node"]
  }
}
EOF
```

1.3. Configure environment variables

Create an `.env` file in the project directory:

Shell

```
touch .env
```

Then, add your credentials. Replace `YOUR_API_KEY` with your actual Circle Developer API key, `YOUR_ENTITY_SECRET` with your entity secret (64 lowercase alphanumeric characters), and `YOUR_EVM_WALLET_ADDRESS` and `YOUR_SOLANA_WALLET_ADDRESS` with the wallet addresses you control through Circle Wallets. You can fetch the addresses from the **Circle Developer Console** or the **list wallets** endpoint:

.env

```
CIRCLE_API_KEY=YOUR_API_KEY
CIRCLE_ENTITY_SECRET=YOUR_ENTITY_SECRET
EVM_WALLET_ADDRESS=YOUR_EVM_WALLET_ADDRESS
SOLANA_WALLET_ADDRESS=YOUR_SOLANA_WALLET_ADDRESS
```

Edit `.env` files in your IDE or editor so credentials are not leaked to your shell history.

Step 2. Bridge USDC

This step shows you how to set up your script, execute the bridge transfer, and check the result.

›

2.1. Create the script

Create an `index.ts` file in the project directory and add the following code. This code sets up your script and bridges 1.00 USDC from Solana Devnet to Arc Testnet.

> Using a different blockchain as the source or destination? Change the `chain` values in `kit.bridge()` and ensure your source wallet has USDC and both wallets have native tokens.

TypeScript

```typescript
// Import App Kit and the Circle Wallets adapter
import { AppKit } from "@circle-fin/app-kit";
import { createCircleWalletsAdapter } from "@circle-fin/adapter-circle-wallets
import { inspect } from "util";

// Initialize the SDK
const kit = new AppKit();

const bridgeUSDC = async (): Promise<void> => {
  try {
    // Set up the Circle Wallets adapter instance, works for both ecosystems
    const adapter = createCircleWalletsAdapter({
      apiKey: process.env.CIRCLE_API_KEY!,
      entitySecret: process.env.CIRCLE_ENTITY_SECRET!,
    });

    console.log("---------------Starting Bridging---------------");

    // Use the same adapter for the source and destination blockchains
    const result = await kit.bridge({
      from: {
        adapter,
        chain: "Solana_Devnet",
        address: process.env.SOLANA_WALLET_ADDRESS!, // Solana address (develo
      },
      to: {
        adapter, // Use the same adapter instance
        chain: "Arc_Testnet",
        address: process.env.EVM_WALLET_ADDRESS!, // EVM address (developer-co
      },
      amount: "1.00",
    });

    console.log("RESULT", inspect(result, false, null, true));
  } catch (err) {
    console.log("ERROR", inspect(err, false, null, true));
  }
};

void bridgeUSDC();
```

You can customize your bridges to **collect a fee**, **use the CCTP Forwarding Service**, or **estimate gas and provider fees** before bridging. Proceed only if the cost works for you.

›

2.2. Run the script

Save the `index.ts` file and run the script in your terminal:

```
npx tsx --env-file=.env index.ts
```

2.3. Verify the transaction

After the script finishes, find the returned `steps` array in the terminal output. Each transaction step includes an `explorerUrl`. Use that link to verify that the USDC amount matches the amount you bridged.

The following code is an example of how an `approve` step might look in the terminal output. The values are used in this example only and are not a real transaction:

Shell

```
steps: [
  {
    name: "approve",
    state: "success",
    txHash: "0xdeadbeefcafebabe1234567890abcdef1234567890abcdef1234567890abcd"
    data: {
      txHash:
        "0xdeadbeefcafebabe1234567890abcdef1234567890abcdef1234567890abcd",
      status: "success",
      cumulativeGasUsed: 17138643n,
      gasUsed: 38617n,
      blockNumber: 8778959n,
      blockHash:
        "0xbeadfacefeed1234567890abcdef1234567890abcdef1234567890abcdef12",
      transactionIndex: 173,
      effectiveGasPrice: 1037232n,
      explorerUrl:
        "https://testnet.arcscan.app/tx/0xdeadbeefcafebabe1234567890abcdef1234
    },
  },
];
```

Was this page helpful? Yes No

Bridge between Solana and EVM Collect custom bridge fees

Arc homepage

Powered by mintlify

By using the Arc Network Testnet, you agree to the Terms of Service and Privacy Policy.